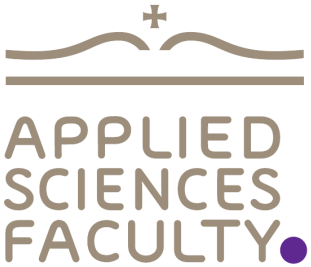
C++ — лекція 24. Алгоритми (Draft!)





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Алгоритми

- Завдяки концепції ітераторів алгоритми в C++ відділені та незалежні від контейнерів.

Типова операція — вивести всі елементи контейнера на екран:

```
vector<int> v1;
v1.push_back(10);
v1.push_back(-11);
v1.push_back(12);
v1.push_back(-13);
v1.push_back(14);

for(vector<int>::iterator itr = v1.begin();
    itr != v1.end(); ++itr )
{
    cout << *itr << "\t";
}
```

Тобто, детально розписуємо комп'ютеру — береш елемент, виводиш, якщо він не останній – продовжуєш.

Однак, наше вихідне твердження оперує іншим, груповим, поняттям — вивести **всі** елементи.
Для реалізації потрібної операції, саме в таких термінах, існує спеціальний алгоритм, `for_each`:

```
void print_int(const int x)
{
    cout << x << "\t";
}
  .........
int main()
{
  .........
    for_each(v1.begin(), v1.end(), print_int );
  .........
}
```

Відповідно до ідеології C++, краще зробити так:

```
template<class T>
void print_val(T x)
{
    cout << x << "\t";
}
  .........
int main()
{
  .........
    for_each(v1.begin(),v1.end(), print_val<int>);
  .........
}
```

Іншою популярною задачею є пошук елемента в контейнері:

```
int main()
{
  .........
     vector<int>:: iterator  pos12 = find(v1.begin(), v1.end(),  12);
     if (pos12!=v1.end() )
          cout << *pos12 << endl;
  .........
}
```

Його вдосконалена форма — пошук елементів, що задовольняють певному критерію:

```
template<class T>
bool is_negative(T x)
{
    return x<0;
}
  .........
int  main()
{
  .........
    vector<int>:: iterator  posm =
          find_if(v1.begin(), v1.end(),  is_negative<int>);
    if (posm!=v1.end() )
        cout << *posm << endl;
  .........
}
```

Якщо є задача підрахувати всі негативні числа, можна реалізувати її за допомогою `find_if`, кожен раз продовжуючи пошук з позиції після знайденого:

```
int counter = 0;
for(vector<int>::iterator itr=v1.begin(); itr != v1.end(); )
{
    itr = find_if( itr ,v1.end(), is_negative<int>);
    if( itr != v1.end() )
    {
        ++counter;
        ++itr;
    }
}
```

Однак це довго, неелегантно, вразливо до помилок і суперечить духу C++.

Для такої поширеної задачі є спеціалізований алгоритм:

```
int counter =
    count_if(v1.begin(), v1.end(), is_negative<int>);
```

Алгоритми, що не модифікують послідовність:

- for_each — виконати дію для всіх елементів контейнера.
- find, find_if — знайти елемент.
- find_first_of — знайти перший елемент зі списку.
- find_end — знайти останнє входження підпослідовності.
- adjacent_find — знайти рівні сусідні елементи.
- count, count_if — підрахувати кількість входжень.
- mismatch — знайти місце, з якого послідовності починають відрізнятися.
- equal — порівняти дві послідовності.
- search — пошук послідовності.
- search_n — пошук вказаної кількості однакових елементів.

Якщо б існували тільки такі алгоритми — це було б нецікаво.

Сортування:

```
for_each(v1.begin(),v1.end(), print_val<int>);
sort(v1.begin(), v1.end());
cout << endl;
for_each(v1.begin(),v1.end(), print_val<int>);
cout << endl; cout << endl ;
```

дасть:

```
10      -11     12      -13     14
-13     -11     10      12      14
```

Виконати певну дію над всіма елементами

```
template<class T>
T negate_val(T x)
{
    return -x;
}
  .........
int main()
{
  .........
    transform(v1.begin(),v1.end(), v1.begin(), negate_val<int>);
    for_each(v1.begin(),v1.end(), print_val<int>);
    cout << endl; cout << endl ;
  .........
}
```

Алгоритми, що модифікують послідовність:

- copy — копіювати з послідовності в послідовність.
- transform — викликати певну функцію для кожного елемента, трансформуючи послідовність.
- replace, replace_if, replace_copy, replace_copy_if — заміна значень в послідовності.
- fill, fill_n — заповнити послідовність значенням.
- generate, generate_n — генерувати значення для послідовності за допомогою функції.
- remove, remove_if, remove_copy, remove_copy_if — видалення елементів послідовності[1].
- unique, unique_copy — видалити послідовні дублювання з послідовності.
- reverse, reverse_copy — змінити порядок елементів у послідовності на протилежний.
- rotate, rotate_copy — зсувати елементи по колу.
- random_shuffle — перемішати елементи випадковим чином.
- partition, stable_partition — розташувати елементи так, що спочатку йдуть ті, які задовольняють вказаному критерію, потім ті, що не задовольняють.
- sort, stable_sort, partial_sort, partial_sort_copy, nth_element — різноманітні засоби сортування.
- ~~Багато інших алгоритмів — з~~лиття, бінарний пошук, тощо.

[1] Так як функція має справу лише з ітераторами, не з контейнерами, то фактично нічого не видаляє, лише пересортовує елементи так, що ті, які слід залишити, будуть на початку. Повертає ітератор, з якого місця слід видаляти. Тоді видалення зведеться до відкидання всіх елементів після нього.

- https://danlark.org/2022/04/20/changing-stdsort-at-googles-scale-and-beyond/ — про особливості реалізації сортування. Російський переклад: https://habr.com/ru/post/662181/.
- https://habr.com/ru/articles/762554/, https://habr.com/ru/articles/763024/ — візуалізація роботи стандартних алгоритмів.
- https://mariusbancila.ro/blog/2025/09/02/what-is-the-cpp-nth_element-algorithm/